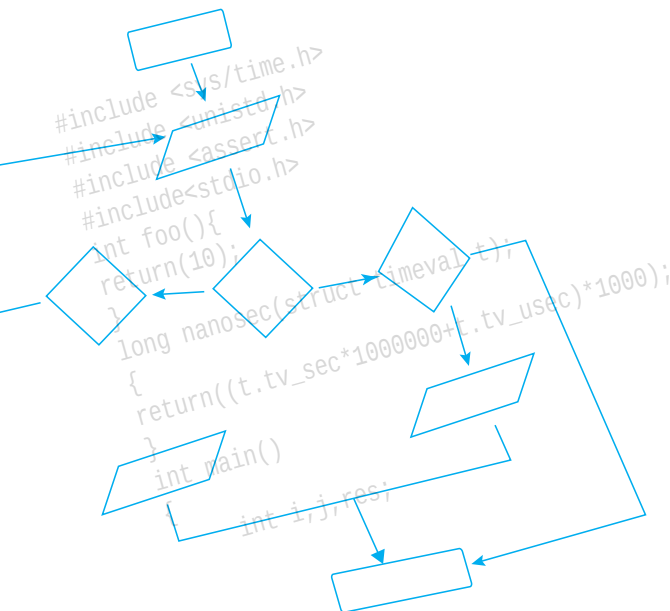


The Core Connection: System and Function Calls Explored

Let's unveil the profound impact of system and function calls, demystifying the interplay between software and hardware that happens in nanoseconds.



In the field of open source, most primary operating systems are *nix based. These operating systems are crucial when it comes to software engineering. Even in interviews with top-tier tech companies, operating systems are almost always integral to the discussion.

In this article, we delve into one of the pivotal aspects of operating systems: system and function calls. Essentially, our goal is to understand the intricacies of what occurs when any action is performed on our computer system. We seek to understand how tasks are executed and explore the various methods used for these executions.

Before we proceed, let us understand the concept of the kernel in an operating system. The kernel is the central most

part of the operating system. It is a protected area of the OS that is inaccessible to all users. It is the kernel that maintains direct contact with the computer's hardware.

Now let us define a system call. When any process on your computer system, such as running a program, requires hardware access, it will communicate or request the kernel to execute the action required. Basically, the process sends a request to the kernel to fulfil a specific task. During this time, the machine gets temporary privileged access to the kernel, if the call is permitted. It performs the required task, and we then go back to the user privileges. This transition is called context switching.

fork() and exec()

Two of the most popular system calls are *fork()* and *exec()*.

Fork is used to create a new process, while exec helps associate and run a process with a task. Now what is a function call? A function call invokes a set of statements defined, but in the user space, not the kernel space. If a function call requires hardware utilisation, it will have to go through the kernel through a system call only.

Now obviously, invoking a system call takes longer than a function call. This is because, when a system call is made, the context should first change from user to kernel, execute the task, and then switch back to the user context. This will take much longer than a simple function call.

Open up a new text editor and type the following code:

```
#include <sys/time.h>
#include <unistd.h>
#include <assert.h>
#include <stdio.h>
int foo(){
    return(10);
}
long nanosec(struct timeval t);
{
    return((t.tv_sec*1000000+t.tv_usec)*1000);
}
int main()
{
    int i,j,res;
```